Token Detail Record

Mediation Output Specification

> Format, content, generation and delivery of Token Detail Records produced by the AgentU Proxy of Record, and the interface presented to consuming systems. Record format version 1.7.

DOCUMENT NUMBER	AGU-TDR-001
ISSUE	1 · August 2026
RECORD FORMAT VERSION	1.7
NETWORK ELEMENT	AgentU Proxy of Record (POR) — ai-proxy
SUPERSEDES	Format versions 1.0 through 1.6
STATUS	In service

Contents

1. SCOPE

1.1 Scope of this document

This document specifies the format, content, generation and delivery of **Token Detail Records (TDRs)** produced by the AgentU Proxy of Record. It is the interface specification between the network element and any downstream system that consumes those records.

This document specifies **what the network element emits**. It does not specify rating logic, downstream presentation, or the internal behaviour of the network element beyond what is observable in the record.

1.2 The record

A TDR describes one complete round trip: the prompt sent to the provider and the response returned from it, recorded as a single unit. Exactly one TDR is generated per completed request traversing the POR.

The record is therefore not written until the round trip reaches a terminal state. Both directions are present on the same record: input and output token counts, input and output character counts, input and output rates, and the input and output components of the charge. No join is required to price an event, and no record exists for a request still in flight.

> **NOTE**
>
> **This is a design decision, not an accident of implementation.** Separate records per direction — one for the prompt, one for the response — were considered and rejected. A single round-trip record means the two halves of an event cannot be separated, cannot be counted twice, and cannot arrive out of order or one without the other. It also means the summation control (field 50) can carry the constant `1` and be summed directly, which a paired-record design would not permit.
>
> A consumer shall treat one record as one chargeable event in full. It shall not expect a matching counterpart record for the opposite direction, because none is generated.

A TDR contains:

- Identification of the record and the request it describes.
- The attribution hierarchy under which the request was made.
- The routing decision and the disposition of the request.
- Quantities reported by the provider, for both directions of the round trip.
- Quantities measured by the POR, for both directions of the round trip.
- The rate card in force, and the resulting charge, split into its input and output components.
- Correlation keys linking the request to a larger interaction.

A TDR does not contain prompt content, completion content, error text, or credential material. See §5.13.

1.3 Terminology

Term	Meaning
TDR	Token Detail Record. The unit record specified herein.
POR	Proxy of Record. The network element generating TDRs.
Event	One inference request traversing the POR and reaching a terminal state.
Collector	The POR instance that generated a record. Identified by `server_id`.
Ledger date	The UTC calendar date to which an event is assigned for accounting.
Seal	The operation that closes a set of buffered records into an immutable file.
Ship	The operation that transfers a sealed file to the archive.
Provider	The upstream model vendor terminating the request.
Field	One of the 55 positional elements of a record, as defined in §5.

The record specified herein is a Token Detail Record, and no other term applies to it. A TDR meters token consumption. Token consumption is not a duration, is not derived from one, and is not reported by a uniform mechanism across providers — so a record type defined around measured elapsed time does not describe it and shall not be substituted for it in specification, interface naming, or correspondence.

1.4 Notation used in the field tables

- **Domain** values are **normative**. A value outside its stated domain is non-conformant.
- **Observed** values are **informative**. They describe values present in a production population of approximately 32,000 records. They describe what has been seen; they do not constrain what is permitted.
- Where a field has an enumerated domain, the enumeration is given in full and is marked as such.

2. RECORD GENERATION

2.1 Trigger

Exactly one TDR is generated when an inference request traversing the POR reaches a terminal state. Terminal states are enumerated at field 21.

2.2 Non-generation

A TDR is **not** generated where:

- The request is refused before provider dispatch by admission control. No chargeable event occurred.
- Authentication fails at the perimeter.

2.3 Duplicate suppression

Each event carries a unique `request_id` (field 2). Where transport-layer redelivery causes an event to be presented to the record generator more than once, the record is emitted once and field 22 (`redelivery`) is set on the redelivered instance.

Field 50 (`events`) is the summation control. It carries the constant `1` on every record. An aggregate over a TDR population is computed by summing field 50, not by counting rows.

3. RECORD STRUCTURE

3.1 Physical format

Attribute	Value
Encoding	UTF-8, no byte order mark
Record delimiter	CR LF (`0x0D 0x0A`)
Field delimiter	COMMA (`0x2C`)
Quoting	Minimal — fields quoted only where they contain delimiter, quote, CR or LF
Quote character	DOUBLE QUOTE (`0x22`), doubled for literal
Header	Present, first record of every file, field names in order
Field count	55, fixed
Record length	Variable
NULL representation	Empty field. See §3.3.

Records are variable-length. A parser that assumes fixed offsets will fail. Parse by field position after delimiter resolution, honouring quoting.

Embedded CR LF is legal inside a quoted field. Counting `0x0A` occurrences does not yield a record count. Record counting requires a conformant CSV parser.

3.2 Field ordering and stability

Field order is **normative**. Downstream systems may address fields by ordinal position.

Within a format version, field order is frozen. Between versions:

- Fields may be **appended** at the end.
- Fields may be **replaced in position** by a field of the same semantic role.
- Fields **shall not** be reordered or removed without a format version increment.

3.3 Empty field semantics

An empty field means **"not measured"** or **"not applicable to this event"**. It does not mean zero.

This distinction is normative. A record with an empty `char_in` (field 41) was generated by a path that did not perform character measurement; a record with `char_in` = `0` was measured and found to contain zero characters.

3.4 Free-text field protection

Fields of type **A** are subject to a **reversible prefix guard**. Where a value begins with `=`, `+`, `-`, `@`, TAB, CR, or an apostrophe, a single apostrophe (`0x27`) is prefixed on output.

The transform is a bijection: a reader removes one leading apostrophe if and only if the next character is one of the marker set. **Numeric and monetary fields are not guarded**, so a negative value is never altered by this mechanism.

4. FIELD TYPES

Every field carries both a **type code**, used as shorthand throughout §5, and a **data type**, which is the type a conforming consumer shall use when loading the field into a typed store.

Code	Name	Data type	Domain as rendered in the record
A	Alphanumeric	`TEXT`	UTF-8 text, variable length. Prefix guard per §3.4.
N	Numeric	`BIGINT`	Unsigned decimal integer. No sign, no grouping separators, no decimal point. Domain `0 .. 2^63-1`.
M	Monetary	`NUMERIC(20,12)`	Fixed-point decimal, exactly 12 decimal places. See §4.1.
T	Timestamp	`TEXT` (ISO 8601)	ISO 8601 with UTC offset, microsecond precision. See note below.
D	Date	`DATE`	ISO 8601 calendar date, `YYYY-MM-DD`.
B	Boolean	`BOOLEAN`	Literal `true` or `false`, lower case. No other value is conformant.
H	Hash	`CHAR(64)`	Exactly 64 lower-case hexadecimal characters, or empty.

> **NOTE**
>
> **Type T is carried as text, not as a native timestamp.** The record preserves the offset and microsecond precision exactly as generated. A consumer loading field 3 into a native timestamp type shall use a type that preserves time zone and microsecond precision; a type that truncates to seconds, or that discards the offset, is not conformant for this field.
>
> **Data types are given in ANSI SQL notation** for definiteness. A consumer using another type system shall select the nearest type that preserves the stated precision, scale and range without loss. `BIGINT` and `NUMERIC(20,12)` are precision requirements, not implementation suggestions.

4.1 Monetary field representation

All monetary fields are rendered as fixed-point decimal with **exactly twelve decimal places**, quantized half-up, corresponding to a `NUMERIC(20,12)` column.

```
0.000004200000        four point two microdollars
0.000000000000        zero, measured
                      empty — not applicable
```

Monetary fields shall be parsed into a decimal type, never a binary floating-point type. Individual events are routinely priced below one microdollar.

Attribute	Value
Precision	20 significant digits
Scale	12 decimal places, always present
Sign	Unsigned. A negative value is non-conformant.
Currency	USD. Not carried in the field.
Exponent notation	Never emitted
Conformant pattern	`^[0-9]+\.[0-9]{12}$`, or empty

5. RECORD LAYOUT

Req — **M** mandatory, always populated · **C** conditional, populated where applicable · **O** optional, populated where available.

5.1 Record identification (fields 0–4)

#	Field	Type	Data type	Req	Domain and values	Description
0	`server_id`	A	`VARCHAR(64)`	**M**	`^[A-Za-z0-9][A-Za-z0-9._-]{0,63}$`. Maximum 64 characters.	Identifier of the collector that generated this record. Also appears in the archive key and filename (§7).
1	`tdr_schema`	A	`VARCHAR(8)`	**M**	`1.7` for records conforming to this document.	Format version of this record. Present on every record so a mixed-version population remains interpretable.
2	`request_id`	A	`TEXT`	**M**	UUID or provider-assigned identifier. Unique within the archive.	Identifier of the event. The correlation key to all other systems.
3	`timestamp`	T	`TEXT`	**M**	ISO 8601, UTC offset, microsecond precision. Example `2026-08-08T01:48:07.123456+00:00`.	Event time.
4	`ledger_date`	D	`DATE`	**M**	`YYYY-MM-DD`. Always equals the UTC date component of field 3.	Calendar date to which the event is assigned. Derived from field 3, never from the time of file generation. See §6.3.

5.2 Attribution (fields 5–13)

Fields 5 through 8 form a hierarchy, broadest to narrowest. An empty attribution field means that level was not established for this event.

#	Field	Type	Data type	Req	Domain and values	Description
5	`client_id`	A	`TEXT`	**M**	Free text. Observed: `ai-proxy`, `dashboard`, `tauric-hedge`.	Top-level attribution. The tenant.
6	`subclient_id`	A	`TEXT`	**C**	Free text.	Second-level attribution.
7	`app_id`	A	`TEXT`	**C**	Free text.	Third-level attribution. Application or workload.
8	`account_id`	A	`TEXT`	**C**	Free text.	Fourth-level attribution.
9	`account_name`	A	`TEXT`	**M**	Free text.	Human-readable account label.
10	`key_id`	A	`TEXT`	**C**	Credential identifier, prefix `cpk_`. Identifier only.	Credential presented. No credential material appears in any field.
11	`usr`	A	`TEXT`	**O**	Free text, caller-supplied.	End-user identifier.

#	Field	Type	Data type	Req	Domain and values	Description
12	`department`	A	`TEXT`	**O**	Free text, caller-supplied.	Cost centre.
13	`project`	A	`TEXT`	**O**	Free text, caller-supplied. Segments shall not contain `/`.	Attribution tag orthogonal to the hierarchy. A project may span clients and is **not** a level of fields 5–8.

5.3 Routing and disposition (fields 14–23)

#	Field	Type	Data type	Req	Domain and values	Description
14	`provider`	A	`TEXT`	**M**	Observed: `Alibaba`, `Amazon`, `Anthropic`, `Cohere`, `DeepSeek`, `Google`, `Mistral`, `Moonshot`, `OpenAI`, `Perplexity`, `xAI`. Not case-normalised.	Upstream vendor terminating the request.
15	`backend_id`	A	`TEXT`	**C**	Free text.	Specific backend instance selected.
16	`key_source`	A	`TEXT`	**C**	Enumerated. Not populated in the current release.	Provenance of the credential used upstream.
17	`geo`	A	`TEXT`	**C**	Region identifier, e.g. `us-east-1`.	Serving region.
18	`routing_reason`	A	`TEXT`	**C**	Free text. Diagnostic.	Why this backend was selected.
19	`model_id`	A	`TEXT`	**M**	Catalog model identifier.	Model **requested** by the caller.
20	`model_used`	A	`TEXT`	**M**	Catalog model identifier.	Model that **served** the request. May differ from field 19 under fallback or alias resolution.
21	`status`	A	`TEXT`	**M**	**Enumerated, complete: `success`, `error`, `interrupted`.**	Terminal disposition of the event.
22	`redelivery`	B	`BOOLEAN`	**M**	`true` or `false`.	`true` where this event was presented to the record generator more than once by the transport. See §2.3.
23	`source`	A	`TEXT`	**C**	**Enumerated: `proxy`, `dashboard`.**	Ingress lane that admitted the event.

> **NOTE**
>
> **Field 14 is not case-normalised.** The production population contains both `Anthropic` and `anthropic`, `OpenAI` and `openai`, `DeepSeek` and `deepseek`. A consuming system grouping by provider shall fold case, or it will report one vendor as two.
>
> **Field 21 has three values, not two.** `interrupted` denotes an event terminated before a normal terminal state was reached. A consumer treating `status` as a two-valued flag will misclassify these records.

5.4 Performance (fields 24–25)

#	Field	Type	Data type	Req	Domain and values	Description
24	latency_ms	N	BIGINT	C	Unsigned integer, milliseconds. Observed 0 .. 711868.	Total elapsed time.
25	overhead_ms	N	BIGINT	C	Unsigned integer, milliseconds. Where both are present, ≤ field 24.	Time attributable to the POR. Provider time is field 24 less field 25.

5.5 Quantities reported by the provider (fields 26–30)

These five fields carry the token counts returned by the provider in the response envelope, recorded as received.

#	Field	Type	Data type	Req	Domain and values	Description
26	input_tokens	N	BIGINT	C	Unsigned integer. Observed 0 .. 514517.	Input tokens. **Excludes tokens served from cache**, which appear in field 28.
27	output_tokens	N	BIGINT	C	Unsigned integer. Observed 0 .. 14904.	Output tokens.
28	cache_read_tokens	N	BIGINT	C	Unsigned integer.	Input tokens served from provider cache.
29	cache_write_tokens	N	BIGINT	C	Unsigned integer.	Tokens written to provider cache.
30	reasoning_tokens	N	BIGINT	C	Unsigned integer.	Reasoning tokens. Metered but not separately rated; provider practice is to include these in field 27 for billing.

> **NOTE**
>
> **Field 26 is not the total input volume.** A low value in field 26 accompanied by a high value in field 28 is normal cache behaviour. Any computation over input volume shall consider fields 26 and 28 together.

5.6 Rating inputs (fields 31–35)

#	Field	Type	Data type	Req	Domain and values	Description
31	rate_version	A	TEXT	C	Opaque token. Observed forms: <YYYYMMDD>T<HHMMSS>-<4 hex>, and the literal seed.	Identifier of the rate card in force at event time. **Opaque** — not to be parsed or ordered, and comparable only for equality.
32	rate_input_per_1m	M	NUMERIC(20,12)	C	Monetary per §4.1.	Input rate, per 1,000,000 tokens.
33	rate_output_per_1m	M	NUMERIC(20,12)	C	Monetary per §4.1.	Output rate, per 1,000,000 tokens.

#	Field	Type	Data type	Req	Domain and values	Description
34	rate_cache_read_per_1m	M	NUMERIC(20,12)	C	Monetary per §4.1.	Cache-read rate, per 1,000,000 tokens.
35	rate_cache_write_per_1m	M	NUMERIC(20,12)	C	Monetary per §4.1.	Cache-write rate, per 1,000,000 tokens.

Rates are carried on the record so that a record remains rateable without reference to external state at any future date. A rate card revision does not alter records already generated.

5.7 Charges (fields 36 and 44)

#	Field	Type	Data type	Req	Domain and values	Description
36	cost_out	M	NUMERIC(20,12)	M	Monetary per §4.1. Unsigned.	Output charge. Output tokens × output rate, quantized once to 12 decimal places.
44	cost_in	M	NUMERIC(20,12)	M	Monetary per §4.1. Unsigned.	Input charge. Ledger total less field 36, clamped at zero.

Observed total charge per event: `0.000000000000 .. 0.838691250000`.

> **NOTE**
>
> **The split is a residual.** Field 44 is derived by subtraction so that field 36 plus field 44 equals the ledger total for the event exactly, with no rounding residue. The two fields are non-adjacent for reasons of format history (§10).
>
> The clamp at zero applies to backends with a zero total and a non-zero rate card, where a naive residual would be negative. A negative total is refused at generation and does not appear in the archive.

5.8 Quantities measured by the POR (fields 41–43)

These three fields carry character counts measured by the POR on payloads it handled directly.

#	Field	Type	Data type	Req	Domain and values	Description
41	char_in	N	BIGINT	C	Unsigned integer. Observed `2 .. 445191`. Empty where the path did not measure.	Characters submitted.
42	char_out	N	BIGINT	C	Unsigned integer.	Characters returned.
43	context_char_in	N	BIGINT	C	Unsigned integer. Where present, ≤ field 41.	Characters attributable to conversational context rather than the current turn.

> **NOTE**
>
> Empty and zero are distinct in these fields and shall not be conflated. See §3.3.

5.9 Estimated decomposition (fields 45–49)

These fields carry an estimated split of input volume between the current turn and carried context, for providers that do not decompose usage. **They are estimates and are not measurements.**

#	Field	Type	Data type	Req	Domain and values	Description
45	`est_user_tokens_in`	N	`BIGINT`	**O**	Unsigned integer.	Estimated tokens attributable to the current turn.
46	`est_context_tokens_in`	N	`BIGINT`	**O**	Unsigned integer.	Estimated tokens attributable to carried context.
47	`est_user_tokens_in_cost`	M	`NUMERIC(20,12)`	**O**	Monetary per §4.1.	Estimated charge for field 45.
48	`est_context_tokens_in_cost`	M	`NUMERIC(20,12)`	**O**	Monetary per §4.1.	Estimated charge for field 46.
49	`est_version`	A	`VARCHAR(16)`	**O**	Opaque token. Current value `cpt-1`.	Estimator version. **Estimates produced by differing versions are not comparable.**

5.10 Payload digests (fields 37–38)

#	Field	Type	Data type	Req	Domain and values	Description
37	`wrap_inbound_sha256`	H	`CHAR(64)`	**C**	64 lower-case hexadecimal characters, or empty.	SHA-256 of the inbound payload as retained in the protected store.
38	`wrap_outbound_sha256`	H	`CHAR(64)`	**C**	64 lower-case hexadecimal characters, or empty.	SHA-256 of the outbound payload as retained.

> **CAUTION**
>
> These fields carry a digest, never content. The digest is computed over exactly the octets the protected store retains, including any truncation and control-character removal applied on storage.
>
> **Empty means no payload was retained. Empty is not the digest of the empty string.**
>
> A deterministic digest is pseudonymous, not anonymous, and is appropriate only for high-entropy payloads.

5.11 Diagnostics (fields 39–40)

#	Field	Type	Data type	Req	Domain and values	Description
39	`app_version`	A	`TEXT`	**O**	Free text, caller-supplied.	Application version.
40	`error_code`	N	`INTEGER`	**C**	Unsigned integer. Observed: `400`. Populated where field 21 is `error` and the provider returned a numeric status.	Numeric class of the failure. Code only.

NOTE

No error text appears in this record, in any form. Provider error bodies may echo caller-supplied content and are retained only in the protected store. Field 40 carries the numeric class of the failure; the body does not cross.

Field 40 is sparsely populated. In the production population it is present on 2 records of approximately 36,300. A consumer shall not infer the disposition of an event from the presence or absence of field 40; field 21 (`status`) is the authoritative disposition.

5.12 Summation and correlation (fields 50–54)

#	Field	Type	Data type	Req	Domain and values	Description
50	`events`	N	`SMALLINT`	M	**Constant 1.** No other value is conformant.	Summation control. See §2.3.
51	`root_id`	A	`TEXT`	C	Identifier matching field 2 of the originating event.	Root of a multi-step interaction.
52	`parent_id`	A	`TEXT`	C	Identifier matching field 2 of another event.	Immediate parent event.
53	`actor_id`	A	`TEXT`	C	Free text.	Acting agent where the event was machine-originated.
54	`leg_type`	A	`TEXT`	C	Enumerated. Observed: `sync`.	Leg classification within a multi-step interaction.

NOTE

Fields 51–54 permit reconstruction of a multi-step interaction tree. **Absence does not imply a single-step interaction**; it implies the correlation was not captured.

5.13 Content excluded from the record

The following shall never appear in a TDR, as a field or within a field:

- Prompt or completion content, in whole or in part
- Provider error bodies or diagnostic text
- Credential material of any kind
- Any field named `prompt_preview`, `wrapped_inbound`, `wrapped_outbound`, or `error_detail`

The generator enforces this by allowlist: a field not named in §5 is not emitted, and a record that cannot be rendered within the allowlist is diverted (§9.2) rather than emitted in reduced form.

6. FILE ORGANISATION

6.1 Generation triggers

A file is sealed on the **first** of:

Trigger	Condition	Default
Count	Buffered records reach the threshold	1,000 records
Time	Oldest buffered record reaches the interval	900 seconds
Date roll	Buffered records span two ledger dates	—

> NOTE
>
> **The count trigger is a threshold, not a slice width.** The buffer is examined at a finite interval and may exceed the threshold before examination. When the trigger fires, the **entire buffer** is sealed, and a file may therefore contain more than the threshold. A conforming reader shall not assume any maximum record count per file. Observed: a file of 1,033 records produced by a 1,000-record threshold.

6.2 File content constraints

Constraint	Value
Distinct `ledger_date` values per file	Exactly 1
Distinct `server_id` values per file	Exactly 1
Minimum data records per file	1
Maximum data records per file	Unbounded — see §6.1
Record ordering within a file	Not guaranteed

6.3 Ledger date assignment

`ledger_date` is derived from the event timestamp, **never** from the time of sealing. An event occurring at 23:59:58 and sealed at 00:00:03 is assigned to the earlier date and appears in that date's file. This is the purpose of the date-roll trigger.

7. FILE NAMING AND ARCHIVE LAYOUT

7.1 Archive key

```
<server_id>/<ledger_date>/<filename>
```

7.2 Filename

```
<server_id>_<sequence>_<timestamp>.csv
```

Component	Format	Domain
`server_id`	Per field 0	`^[A-Za-z0-9][A-Za-z0-9._-]{0,63}$`
`sequence`	16 lower-case hexadecimal digits, zero-padded	`0000000000000000` .. `ffffffffffffffff`
`timestamp`	`YYYYMMDDHHMMSS`, UTC	Seal time, not content time

Conformant filenames match:

```
^(?P<server>.+)_(?P<seq>[0-9a-f]{16})_(?P<stamp>\d{14})\.csv$
```

Example:

```
i-005102093c28eb837/2026-08-08/i-005102093c28eb837_0000000000000002_20260808011515.csv
```

7.3 Properties

Property	Statement
Ordering	Lexicographic order equals seal order within a date prefix.
Sequence scope	Per collector, per ledger date. Resets to zero at each date.
Sequence base	**Hexadecimal.** File 10 of a date is `...000000000000000a`.
Key derivation	Derived from the file's own path, never from current configuration.
Redundancy	`server_id` appears in both key prefix and filename, so a file remains self-describing if separated from its path.

8. DELIVERY

8.1 Two-stage delivery

Record generation and archive transfer are performed by **separate processes**:

1. **Seal** — the generating element writes the file to local persistent storage and performs no network operation.
2. **Ship** — an independent process transfers sealed files to the archive on a periodic cycle. Default cycle 60 seconds.

Local storage is the durable buffer. Archive unavailability causes files to accumulate locally and to be transferred on recovery; it does not delay or fail the generating element.

8.2 Transfer properties

Condition	Behaviour
Normal	Write-once. Transfer is conditional on the object key not existing.
Key exists, identical content	Local file released. Transfer is idempotent under retry.
Key exists, differing content	Local file assigned the next free sequence and transferred under the new key. No object is overwritten and no record is discarded.
Transfer fails	Local file retained in place for the next cycle.
File cannot be transferred	Remains queued. There is no local quarantine of untransferable files.

8.3 Delivery guarantees

Property	Guarantee
Loss	No record is released locally before confirmed archive receipt.
Duplication	Prevented by write-once transfer. Retry is idempotent.
Ordering	Not guaranteed between files. Sequence and timestamp permit ordering by the reader.
Latency	Bounded by seal trigger plus transfer cycle.

8.4 Queue age

The integrity indicator for the delivery path is the **age of the oldest untransferred file**. This measure degrades under generation failure, transfer failure, scheduler failure, credential expiry and archive unavailability alike, because all present identically: files cease leaving local storage.

9. EXCEPTION HANDLING

9.1 Classification

Class	Example	Disposition
Record fault	Malformed monetary value; unrenderable field	Record diverted (§9.2). Remaining records in the batch are sealed normally.
Environment fault	Storage full; permission denied; device error	Records retained for retry. Not diverted.

> **CAUTION**
> An environment fault shall never cause record diversion. The records are valid; the equipment is not.

9.2 Diversion

A record that cannot be rendered is written to a diversion store adjacent to the archive queue, is counted, and is logged. It is never discarded silently.

Diversion is bounded: a single unrenderable record within a batch is isolated by bisection, and the remaining records of that batch are sealed and transferred normally. **A batch is a sealing convenience and is not a unit of accounting.**

> **NOTE**
> There is no automatic re-entry from diversion into the archive. A diverted record requires operator action.

9.3 Malformed monetary values

A monetary input that is present but not a decimal value causes **record diversion**. It is not rendered as zero.

9.4 Shutdown

On orderly shutdown the generating element ceases accepting new records, seals the outstanding buffer, and terminates. Records that cannot be sealed are diverted and counted.

10. VERSION HISTORY

Version	Fields	Change
1.5	56	Correlation fields appended (51–54).
1.6	56	`error_detail` replaced in position by `error_sha256`. Provider error text ceased to be archived.
1.7	**55**	`error_sha256` removed entirely. `events` moved from ordinal 51 to 50.

10.1 Note on the 1.7 ordinal shift

Removing a field and shifting subsequent ordinals is **not permitted** under §3.2. It was permitted once, at 1.7, because the archive was empty at the time of the change and no consumer addressed a shifted ordinal. No further ordinal shift will be made without a migration procedure.

11. CONFORMANCE REQUIREMENTS FOR CONSUMING SYSTEMS

A system consuming TDRs shall:

1. Parse with a conformant CSV parser honouring quoting and embedded CR LF. Line counting is not record counting.
2. Parse monetary fields into a decimal type, never a binary floating-point type.
3. Preserve the distinction between empty and zero (§3.3).
4. Sum field 50 for event counts, not count rows.
5. Rate on field 20 (`model_used`), not field 19.
6. Treat fields 31 and 49 as opaque.
7. Parse the filename sequence as base 16.
8. Fold case when grouping by field 14 (`provider`).
9. Treat field 21 (`status`) as three-valued.
10. Exclude fields 45–48 from any amount presented as measured.
11. Verify field 1 (`tdr_schema`) before applying ordinal field addressing.

12. APPENDIX A — SAMPLE RECORD

Header record and one data record. Field delimiters are preserved; the line breaks shown are presentational only.

```
server_id,tdr_schema,request_id,timestamp,ledger_date,client_id,subclient_id,
app_id,account_id,account_name,key_id,usr,department,project,provider,
backend_id,key_source,geo,routing_reason,model_id,model_used,status,redelivery,
source,latency_ms,overhead_ms,input_tokens,output_tokens,cache_read_tokens,
cache_write_tokens,reasoning_tokens,rate_version,rate_input_per_1m,
rate_output_per_1m,rate_cache_read_per_1m,rate_cache_write_per_1m,cost_out,
wrap_inbound_sha256,wrap_outbound_sha256,app_version,error_code,char_in,
char_out,context_char_in,cost_in,est_user_tokens_in,est_context_tokens_in,
est_user_tokens_in_cost,est_context_tokens_in_cost,est_version,events,root_id,
parent_id,actor_id,leg_type
```

```
i-005102093c28eb837,1.7,9c5c0cb5-b76f-4e21-91a3-2f0d7c1e4a88,
2026-08-08T01:48:07.123456+00:00,2026-08-08,ai-proxy,,tdr-golive-load,,AgentU,
cpk_7f2a,,cos,,DeepSeek,,,us-east-1,default,deepseek-v4-flash,
deepseek-v4-flash,success,false,proxy,812,,89,15,,,,20260713T110816-04a2,
0.140000000000,0.280000000000,0.000000000000,0.000000000000,0.000004200000,
,,,,350,40,,0.000007800000,88,,0.000000000000,,cpt-1,1,,,,sync
```

12.1 Field-by-field reading of the sample

#	Field	Value	Reading
0	`server_id`	`i-005102093c28eb837`	Collector instance.
1	`tdr_schema`	`1.7`	Conforms to this document.
2	`request_id`	`9c5c0cb5-…`	Event identifier.
3	`timestamp`	`2026-08-08T01:48:07.123456+00:00`	Microsecond precision, UTC.
4	`ledger_date`	`2026-08-08`	Matches the UTC date of field 3.
5	`client_id`	`ai-proxy`	Top-level attribution.
7	`app_id`	`tdr-golive-load`	Third-level attribution.
14	`provider`	`DeepSeek`	Mixed case as received.
19, 20	`model_id`, `model_used`	`deepseek-v4-flash`	Requested and served agree; no fallback occurred.
21	`status`	`success`	Terminal disposition.
22	`redelivery`	`false`	Presented once.
23	`source`	`proxy`	Ingress lane.
24	`latency_ms`	`812`	Total elapsed.
26, 27	`input_tokens`, `output_tokens`	`89`, `15`	Provider-reported.

#	Field	Value	Reading
28, 29	cache fields	*(empty)*	**Not measured, not zero.**
31	`rate_version`	`20260713T110816-04a2`	Opaque; equality comparison only.
33	`rate_output_per_1m`	`0.280000000000`	Rate in force at event time.
36	`cost_out`	`0.000004200000`	15 tokens at the field 33 rate.
41, 42	`char_in`, `char_out`	`350`, `40`	Measured by the POR.
43	`context_char_in`	*(empty)*	This path does not decompose context.
44	`cost_in`	`0.000007800000`	Residual against the ledger total.
45	`est_user_tokens_in`	`88`	Estimate, not a measurement.
49	`est_version`	`cpt-1`	Estimator version.
50	`events`	`1`	Summation control.
54	`leg_type`	`sync`	Single synchronous leg.

End of document AGU-TDR-001, Issue 1.

END OF DOCUMENT AGU-TDR-001 · ISSUE 1